

Obligatorisk oppgave 5: Programmering med tråder

Andreas Isaksen

September 27, 2024

Oppgave 1

Følgende C-program er gitt:

```
#include <pthread.h>
#include <stdio.h>
#define N_THREADS 5

int value = 0; // Global variabel som deles av alle
               traader

void *count()
{
    int i;

    for (i = 0; i < 100000; i++)
        value++;

    printf("%d ", value);
}

int main ()
{
    int i;
    pthread_t thread[N_THREADS];

    for(i = 0; i < N_THREADS; i++)
        pthread_create(&thread[i], NULL, count, NULL);

    for(i = 0; i < N_THREADS; i++)
        pthread_join(thread[i], NULL);

    printf("\n");
}
```

a) Kjør programmet gitt ovenfor flere ganger. Inkluder utskriften fra fem påfølgende kjøring i besvarelsen din, som et skjermbilde eller ved å gjøre klipp-og-lim av teksten.

Du vil se at tallene som skrives ut av trådene forandrer seg i hver kjøring. Gi en kort forklaring på hvorfor dette skjer.

Svar:

```
[aragnaroht@archlinux part2]$ ./oppg1
237409 250196 255684 260325 356192
[aragnaroht@archlinux part2]$ ./oppg1
```

```
143134 154585 158089 167013 255837
[aragnaro@archlinux part2]$ ./oppg1
105292 127283 138479 224762 236275
[aragnaro@archlinux part2]$ ./oppg1
138606 153951 164168 170912 254716
[aragnaro@archlinux part2]$ ./oppg1
231693 251043 260311 266744 354048
```

Listing 1: shell terminal

Siden det ikke er implementert noen *join* løsning på tråder som blir opprettet i programmet, så ender disse fem trådene med å jobbe over hverandre, fremfor å vente på at den første tråden skal bli ferdig med iterasjonen før den andre starter. Når alle tråder da itererer mot samme globale variabel, så resulterer dette i uforutsigbare utskrifter. Det nærmeste som er forutsigbart her er at den første tråden vil ha den laveste verdien, siden trådene returnerer verdien på sin siste iterasjon og jo senere en tråd starter, jo høyere verdi vil den da returnere.

b) Skriv om koden slik at programmet alltid skriver ut:

```
100000 200000 300000 400000 500000
```

Programmet skal fortsatt starte 5 tråder fra `main()`. Det er bare koden i `main()` som du skal endre litt på, resten av programmet skal være uforandret. Programkoden din skal lagres i en fil med navn `oppgave1_b.c`.

Svar:

Her samlet jeg programmet til en iterering fremfor to separate, slik at for hver tråd som ble opprettet, så ventet neste tråd på at den forgående skal bli ferdig før den starter sin prosess.

Oppgave 2

a) Kompiler programmet gitt ovenfor på studentserveren itstud.hiof.no. Kjør det deretter med time for å måle tidsforbruket når det kjøres på serveren. Inkluder utskriften fra time i besvarelsen.

Svar:

```
andreis@itstud:~/intro20S/oblig5$ time ./oppg2
.
.
.
.
.
.
.
.
.
.
.
1000000000 iterations completed!

real    0m2.574s
user    0m2.568s
sys     0m0.004s
```

Listing 2: shell terminal

b) Kompiler også dette programmet på studentserveren itstud.hiof.no. Husk å bruke opsjonen -pthread til gcc. Kjør deretter programmet med time for å måle tidsforbruket når det kjøres på serveren. Inkluder utskriften fra time i besvarelsen din.

Svar:

```
andreis@itstud:~/intro20S/oblig5$ gcc -pthread -o
  oppg2b oppg2b.c
andreis@itstud:~/intro20S/oblig5$ time ./oppg2b
.
.
.
.
.
.
.
.
.
.
.
```

```
1000000000 iterations completed!  
  
real    0m1.101s  
user    0m2.167s  
sys     0m0.004s
```

Listing 3: shell terminal

c) Gi en forklaring på hvorfor trådprogrammet blir omtrent dobbelt så raskt.

Svar:

Når jeg kjører kommandoen *lscpu* — *more* på serveren kan jeg se at den har to CPUer. Når vi kjører koden på denne måten kan prosessen fordeles over flere prosessor enheter. I dette tilfellet blir den potensielt dobbelt så raskt om programmets prosess blir fordelt på to prosessorer som kjører parallelt.